Báo cáo khảo sát GitHub: Repo giảm token, chi phí và usage LLM

Phiên bản: 1.0 **Ngày snapshot dữ liệu:** 2026-10-28 (theo phản hồi GitHub API trong phiên khảo sát) **Phạm vi:** GitHub repository công khai, mã nguồn mở hoặc có mã nguồn công khai; ưu tiên repo có tác động trực tiếp đến token, chi phí, độ trễ hoặc hiệu suất phục vụ LLM.

> Tóm tắt điều hành. Không thể chứng minh “tất cả repo GitHub” trong một lần crawl: GitHub có hàng triệu repository, kết quả thay đổi theo truy vấn, stars và rate limit. Báo cáo này là **shortlist có phương pháp**, được tìm bằng nhiều truy vấn chủ đề và đối chiếu metadata repo qua GitHub API. Số stars là snapshot, không phải số liệu cố định.

1. Kết luận nhanh

- Giảm prompt/context trực tiếp: microsoft/LLMLingua, zilliztech/GPTCache, aurelio-labs/semantic-router. - **Giảm chi phí bằng routing/gateway/observability:** BerriAI/litellm, Portkey-AI/gateway, langfuse/langfuse, comet-ml/opik, Helicone/helicone. - **Giảm chi phí hạ tầng/inference:** ggml-org/llama.cpp, vllm-project/vllm, sgl-project/sglang, lm-sys/FastChat. - **Giảm context cần gửi bằng retrieval/memory:** mem0ai/mem0, run-llama/llama_index, lancedb/lancedb, stanford-futuredata/ColBERT, getzep/graphiti. - **Tối ưu chất lượng trên mỗi token:** stanfordnlp/dspy, promptfoo/promptfoo, dottxt-ai/outlines, 567-labs/instructor.

Không nên coi “nhiều stars” là bằng chứng tiết kiệm token. Stars chỉ là tín hiệu phổ biến; hiệu quả thực tế phải đo bằng input tokens, output tokens, cache hit-rate, cost/request, latency và chất lượng đầu ra trên workload của bạn.

2. Phương pháp và tiêu chí

2.1. Thu thập

1. Dùng nhiều truy vấn GitHub Search API cho các cụm: token optimization, prompt compression, semantic cache, LLM cost optimization, model routing, context compression, LLM observability, KV cache inference. 2. Bổ sung shortlist các dự án nền tảng phổ biến có cơ chế tác động đến usage/cost. 3. Đối chiếu metadata bằng endpoint công khai GET https://api.github.com/repos/{owner}/{repo}. 4. Sắp xếp giảm dần theo stargazers_count trong snapshot.

2.2. Mức độ liên quan

- Trực tiếp: có tính năng compression, cache, routing, cost tracking hoặc inference efficiency. - **Hỗ trợ:** chủ yếu giảm context, giảm request thừa, cải thiện retrieval/quality hoặc vận hành. - **Gián tiếp:** nền tảng LLM/serving tổng quát; có thể giảm chi phí hạ tầng nhưng không nhất thiết giảm số token.

2.3. Giới hạn

- GitHub API có rate limit; một truy vấn bổ sung trả 403 Forbidden, do đó các repo không có snapshot xác minh không được gán số stars. - Mô tả “giảm token” trong báo cáo là cơ chế kỹ thuật suy ra từ chức năng repo, không phải cam kết tiết kiệm cho mọi workload. - Một số metadata API trả license NOASSERTION; cần đọc file LICENSE của từng repo trước khi dùng thương mại. - Snapshot có thể thay đổi sau khi phát hành báo cáo.

3. Bảng xếp hạng tổng hợp theo stars

Lưu ý thứ tự: Bảng ưu tiên stars nhưng một vài dòng được nhóm lại theo chủ đề; hãy dùng con số stars làm snapshot tham khảo. Với các repo có stars cao nhưng tác động gián tiếp, không nên hiểu là “repo tiết kiệm token tốt hơn” repo chuyên biệt.

4. Phân nhóm theo tính năng

Nhóm A — Prompt và KV-cache compression

Nhóm B — Semantic cache và context/memory

Nhóm C — Retrieval, chunking và grounding

Nhóm D — Routing, gateway và lựa chọn model

Nhóm E — Inference và serving hiệu quả

Nhóm F — Evaluation, observability và structured output

5. Gợi ý stack tối thiểu theo mục tiêu

A. API production, ưu tiên giảm hóa đơn

LiteLLM hoặc Portkey Gateway → semantic router → model rẻ/nhanh cho request đơn giản → model mạnh cho request khó → Langfuse/Helicone/OpenLIT để đo cost và token → GPTCache cho request lặp.

B. RAG tài liệu dài

Unstructured (ETL) → LlamaIndex/LanceDB/ColBERT (retrieval) → tùy chọn LLMLingua (nén context) → promptfoo (đánh giá recall/answer quality).

C. Tự host để giảm chi phí API

llama.cpp cho local/edge hoặc vLLM/ SGLang cho GPU server → quantization phù hợp → benchmark throughput, latency, quality trước khi chuyển production.

D. Agent có hội thoại dài

Mem0 hoặc Graphiti để lưu memory có chọn lọc → semantic cache → observability → đặt giới hạn context và TTL memory.

6. Kế hoạch thử nghiệm đo tiết kiệm

1. Chọn 100–1.000 request đại diện, lưu input/output, model, latency, token và chất lượng chấm bởi rubric. 2. Chạy baseline không tối ưu. 3. Thử từng cơ chế độc lập: cache, routing, compression, retrieval, structured output. 4. Đo các chỉ số: input_tokens/request, output_tokens/request, cost/request, cache_hit_rate, p50/p95 latency, task success, groundedness, retry rate. 5. Chỉ triển khai khi quality không giảm quá ngưỡng chấp nhận và tổng chi phí (kể cả compressor/embedding/DB) thực sự giảm. 6. Kiểm tra riêng dữ liệu nhạy cảm, retention, license và khả năng rollback.

7. Tài liệu nguồn và truy xuất

- GitHub repository pages được liên kết trực tiếp trong từng dòng bảng. - GitHub REST API repository metadata: https://api.github.com/repos/{owner}/{repo}. - GitHub Search API: https://api.github.com/search/repositories. - Các con số stars trong báo cáo là dữ liệu snapshot do API trả về trong phiên khảo sát; cần truy vấn lại trước khi xuất bản hoặc ra quyết định mua sắm.

8. Kết luận

Nếu mục tiêu là giảm token trực tiếp, bắt đầu từ LLMLingua, GPTCache, retrieval và memory chọn lọc. Nếu mục tiêu là **giảm hóa đơn tổng**, ưu tiên gateway/routing + observability trước, vì chúng cho biết tiền đang đi đâu và cho phép dùng model rẻ hơn. Nếu mục tiêu là **giảm cost hạ tầng**, benchmark llama.cpp, vLLM và SGLang trên workload thật. Không có repo nào tự động bảo đảm tiết kiệm; mọi tuyên bố cần được xác nhận bằng benchmark có baseline và quality gate.

Ghi chú biên tập: Tên repo, URL, stars và mô tả giữ nguyên dạng gốc để dễ kiểm chứng; phần diễn giải tiếng Việt là phân tích của báo cáo.

Cập nhật: Khi tái sử dụng, hãy ghi lại timestamp truy vấn, endpoint, commit/release và kết quả benchmark.